Report project isw2
Francesco Astolfi

Link to the repo

Project analized:

Francesco
Astolfi

apache/
bookkeeper

Milestone 1
the dataset

Francesco Astolfi

How the dataset is built :

The dataset is built for each release to enable walk-forward training and to allow users to reconstruct the history and evolution of each bug, including its labeling. Each release analyzes the commits that were pushed to GitHub before the start of the release, and considers all tickets that were published before the end of the release being analyzed.

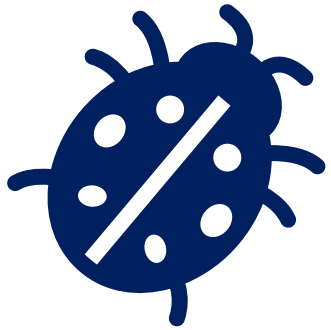
For each project, only the first 33% of the available releases were selected.

However, for smaller projects like BookKeeper, it was necessary to use up to 55% of the releases to ensure sufficient data for analysis.

DATASET

How the bug are selected

Buggyness is assigned based on commits linked to bug-related tickets. If a commit modifies one or more methods and is associated with a bug ticket, those methods are labeled as buggy.
Then labeling procedure propagates buggyness from the injected version(or the evaluated one) up to the fixed version.



DATASET

Wich metrics and why

Features such as nr, age, and nAuth are computed at the class level, while all other features are evaluated at the method level.

- CK metrics (e.g: loc, wmc, fanin, fanout) are extracted using a support library.
- nSmell is computed using a custom PMD runner.

The actionable evaluated feature are all feature that could be easy mesurable and just track a good way of coding, the analisis is focused on find wich of this simple element is more correlated to the buggyness.

DATASET

Some of the introduced improved

- Commit caching was implemented to optimize performance.
- The analysis was executed in parallel across multiple threads.
- A maximum number of classes per commit was analyzed; any excess was cached and assigned to other threads.
- Persistence-related data was encapsulated and hidden from the rest of the project through the use of a DAO
- All constants and path definitions relevant to the project were centralized in a single class to facilitate future reconfiguration.
- All training and test files that have not yet been written are generated.
- During the commit analysis, partial writes may occur to allow better monitoring and tracking of the process.

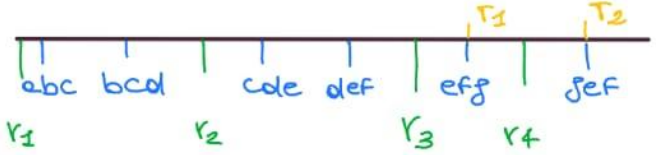
Despite these improvements, the project remains difficult to configure and understand.

Some adverstment

Even if a release contains tickets associated with bugs, it is acceptable that no buggy methods appear in the results. This is because some of those tickets may refer to commits made in the current release, rather than to commits from previous releases.
However, the labeling of buggy methods is based only on the commits pushed to GitHub before the start of the release currently being analyzed. Therefore, if a bug is introduced and fixed within the same release, it may not be labeled as buggy in this context.

Commit
abc → r_1 $foo^{V_1}()$ (I)
bcd → r_1 $foo^{V_2}()$
cde → r_2 $foo1^{V_1}()$ (II)
def → r_2 $foo^{V_3}()$
efg → r_3 $foo^{V_4}()$ (F)
gfe → r_4 $foo^{V_5}()$ (F)

ticket associated commit
- efg → T_1 IV = r_2 FV = r_3
- gfe → T_2 IV = r_1 FV = r_4



dataset r_1 = { }
dataset r_2 = {{ r_1, Foo, V_2, NB }}
dataset r_3 = {{ r_1, Foo, V_2, NB }, { r_2, Foo, V_3, NB }, { r_2, Foo1, V_1, NB }} ticket {1}
dataset r_4 = {{ r_1, Foo, V_2, NB }, { r_2, Foo, V_3, B }, { r_2, Foo1, V_1, NB }, { r_3, Foo, V_4, NB }} ticket {1, 2}
dataset r_5 = {{ r_1, Foo, V_2, B }, { r_2, Foo, V_3, B }, { r_2, Foo1, V_1, B }, { r_3, Foo, V_4, B }, { r_4, Foo, V_5, NB }} ticket {1, 2}

DATASET

Some adverstment

Association of buggyness in a release dataset :

Even if a method is associated with a specific commit, it may also belong to a broader set of methods modified by other commits. This occurs due to a rebalancing of the methods to be processed in the cache. However, this behavior does not affect the labeling of buggyness.
The identification of buggy methods is based on the methods modified in commits linked to bug-related tickets. jGit is used to extract the methods changed in each commit.
Then, buggyness is assigned to the affected version, considering the interval from the injected version to the fixed version.

Approach of evaluation of the Injected Version when it missing:

1. Cold start proportion at the beginning
2. Incremental, each release take the ticket available, opening version before the release starts, evaluate proportion on the ticket that have a valid injected version.
3. Adjust the injected version of the ticket that not have a valid one with proportion

DATASET

Future improvements:

Two approaches:

More precise evaluation of metrics (Age, NR, NAuth) at method level

This choice adds complexity to the project but improves the accuracy of the metrics. The method-level approach is also scalable, allowing for analysis of code evolution on a per-release basis, rather than relying only on cumulative results across all releases.

Simplify the project by evaluating only at the end, maintaining class-level metrics

This option makes the project simpler by analyzing the entire codebase only at the final stage, keeping metrics at class level.

Hybrid approach:

Maintain the final project view, but evaluate only **Age**, **NAuth**, and **NR** metrics commit-by-commit.

DATASET

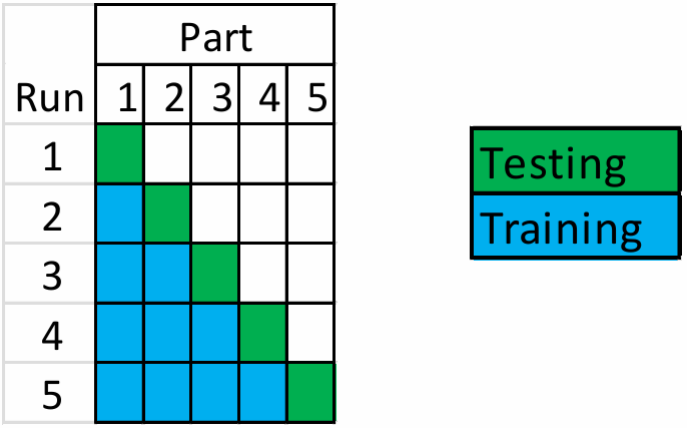
Milestone 2
Compare the 3 classificators

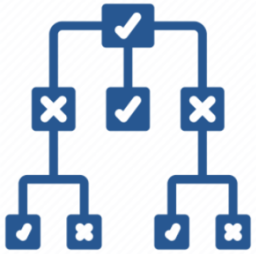
Francesco
Astolfi

The Models

Training testing procedure:

The training process follows a walk-forward approach.
The dataset is maintained in chronological order over time, and the smallest unit used for splitting is the software release.

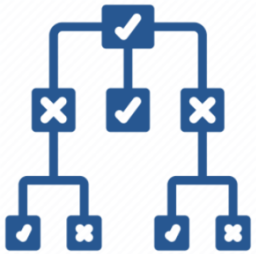




The Models

Important notes about the dataset !

The first available release for training is release 2, as the goal is to verify whether the previous release contains bugs.
For this reason, the commits considered belong to the *previous* release, while the tickets taken into account refer to the *current* release.
Only the final test release utilizes all commits available throughout the project.

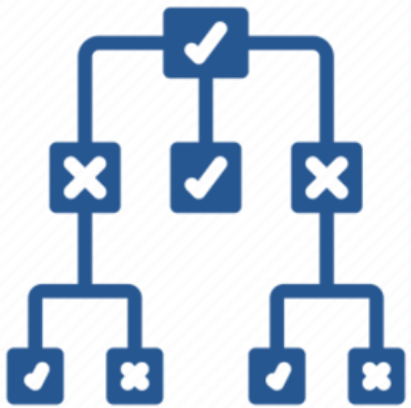


The Models

Models considerated

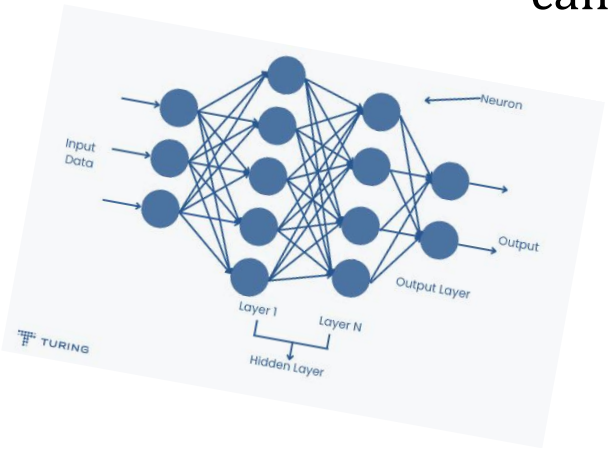
Multi Layer Parceptron:

This is the most sophisticated model, requiring more time and computational resources due to its complexity. It is particularly useful for tasks involving data that do not exhibit a linear correlation and is commonly used to learn how to process and model complex input data.



Statistic Gradient Descendent:

This is the simplest model for a task, but it may underfit more complex problems.
Its main advantage lies in the ease of interpretability, as anyone can easily understand how decisions are made.



Statistic Gradient Descendent:

Random Forest is generally one of the most accurate models across a wide range of scenarios.
It combines the predictions of multiple decision trees to improve overall performance and reduce the risk of overfitting.

Evaluation metrics

Precision :

This metric evaluates how well the model can correctly predict positive instances while discarding the negative ones.
It is generally easier to maximize either this measure or recall individually, but it is not easy to optimize both simultaneously.

Recall:

Instead of precision, recall is used because the "snoring" behavior cannot produce false positives and even if it does, they are less critical than false negatives.
A false negative (i.e., an instance that is actually positive but predicted as negative) is considered more harmful in this context.

Auc :

The Area Under the Curve (AUC) refers to the area under the ROC curve, which plots the recall (true positive rate) on the y-axis and the false positive rate on the x-axis.
The key advantage of this metric is that it is independent of the decision threshold used to distinguish between positive and negative instances.

Y-axis TP/TP+FN

X-axis FP/ FP+TN

Kappa :

The number of time that the model is better than a random model.

The Models

Applied teory during training

Feature selection :
A ranked greedy approach was used, where for each of the n evaluated features,
only p (with $p < n$) were selected based on their ranking to optimize the evaluation process.
This method considers features jointly rather than treating them independently.

Sampling :
If there were more positive instances than negative ones, a portion of the positive instances was discarded in order to balance the number of samples between the two classes.

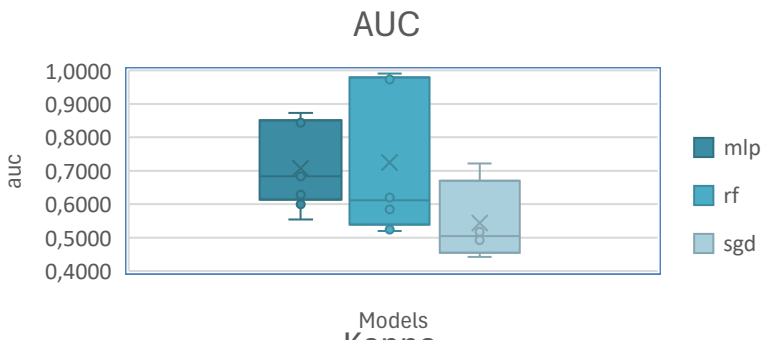
Preprocessing:
From all the methods in the dataset, the identifying features such as MethodName, ClassName, ClassPath, and Release are removed. These features are excluded as they serve only to uniquely identify a method within a specific release and are not useful for training or evaluating predictive models.

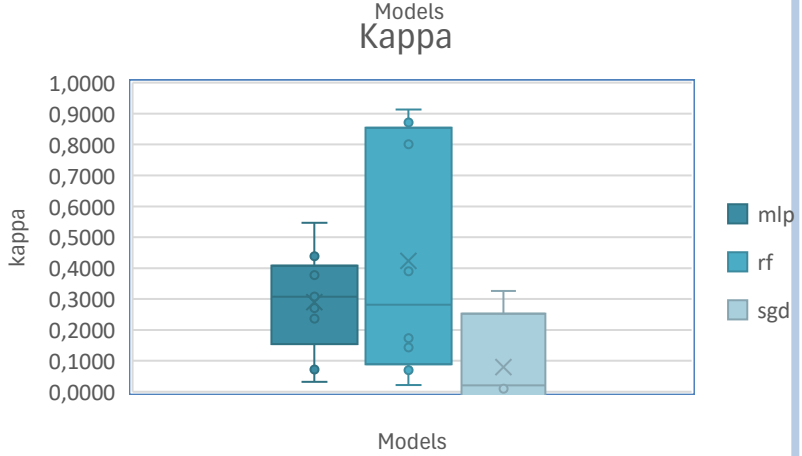
The Models

EXPERIMENTS DONE :

link to the document

	Training set	Training Process	Testing set	Classifier	Feature selection	Sampling	Precision	Recall	Auc	Kappa
1	Openjpa	Walk-Forward	Openjpa	rf	false	true	0,7492	0,9430	0,9728	0,8012
2	Openjpa	Walk-Forward	Openjpa	mlp	false	false	0,3655	0,8458	0,8735	0,3778
3	Openjpa	Walk-Forward	Openjpa	sgd	false	false	0,3595	0,6806	0,7218	0,3264
4	Openjpa	CrossValidation	Openjpa	rf	false	false	0,9680	0,9850	0,9910	0,9129
5	Openjpa	CrossValidation	Openjpa	mlp	false	false	0,8100	0,8090	0,8590	0,4388
6	Openjpa	CrossValidation	Openjpa	sgd	false	false	0,7870	0,9330	0,6380	0,3252
7	Bookkeeper	Walk-Forward	Bookkeeper	rf	false	false	0,8258	0,3624	0,6200	0,3900
8	Bookkeeper	Walk-Forward	Bookkeeper	mlp	false	false	0,5822	0,4130	0,5547	0,3078
9	Bookkeeper	Walk-Forward	Bookkeeper	sgd	true [age]	false	0,2997	0,5234	0,4926	0,0100
10	Bookkeeper	CrossValidation	Bookkeeper	mlp	false	false	0,8160	0,8210	0,8440	0,5472
11	Bookkeeper	CrossValidation	Bookkeeper	rf	false	false	0,9460	0,9460	0,9820	0,8717
12	Bookkeeper	CrossValidation	Bookkeeper	sgd	false	false	0,6940	0,6940	0,5000	0,0000
13	Openjpa	Walk-Forward	Bookkeeper	mlp	false	false	0,5148	0,0403	0,6833	0,0318
14	Openjpa	Walk-Forward	Bookkeeper	mlp	true [age,nr]	false	0,5252	0,4260	0,6282	0,2708
15	Openjpa	Walk-Forward	Bookkeeper	rf	true[age,nr]	false	0,7633	0,1276	0,5847	0,1437
16	Openjpa	Walk-Forward	Bookkeeper	sgd	true[age,nr]	false	0,3235	0,4322	0,5172	0,0316
17	Openjpa	CrossValidation[Model 4 trained]	Bookkeeper	rf	false	false	0,6640	0,6980	0,6040	0,1735
18	Openjpa	CrossValidation[Model 5 trained]	Bookkeeper	mlp	false	false	0,7060	0,9850	0,6000	0,0714
19	Openjpa	CrossValidation[Model 6 trained]	Bookkeeper	sgd	false	false	0,6800	0,7000	0,5160	0,0441
20	Bookkeeper	Walk-Forward	Openjpa	mlp	false	false	0,3752	0,5235	0,6932	0,3288
21	Bookkeeper	Walk-Forward	Openjpa	sgd	true [age]	false	0,1144	0,4809	0,4426	-0,0490
22	Bookkeeper	Walk-Forward	Openjpa	rf	false	false	0,1496	0,4532	0,5196	0,0212
23	Bookkeeper	CrossValidation[Model 10 trained]	Openjpa	mlp	false	false	0,6950	0,7090	0,6300	0,2366
24	Bookkeeper	CrossValidation[Model 11 trained]	Openjpa	rf	false	false	0,6280	0,6570	0,5240	0,0700
25	Bookkeeper	CrossValidation[Model 12 trained]	Openjpa	sgd	false	false	0,7220	1,0000	0,5000	0,0000
	Number of set for bookkeeper : 5					Number of set for openjpa: 12				





Multi Layer Parceptron
Get maybe not the best results but is more stable than the random forest

Experimental discover

The most interesting aspect is observing the in performance when training on OpenJPA and testing on Bookkeeper, and vice versa.

The Models

THE MORE CORRELATED FEATURE TO THE BUGGYNESS

OpenJPA

link to the document

OPENJPA		Smell Correlation	Bug Correlation
1	loc	0.5847	0.1386
2	wmc	0.4341	0.1221
3	assignmentQTY	0.4538	0.1061
4	mathOpQTY	0.2166	0.0463
5	TryCatchQTY	0.1635	0.0737
6	qtyReturn	0.174	0.0868
7	fanin	0.0135	0.0847
8	fanout	0.3762	0.1683
9	age	0.0217	0.1195
10	nAuth	0.041	0.2574
11	nr	0.0568	0.4395
12	nSmell		0.0731

Bookkeeper:

BOOKKEEPER		Bug Correlation
1	loc	0.09747
2	wmc	0.04225
3	assignmentQTY	0.04935
4	mathOpQTY	0.07979
5	TryCatchQTY	0.0954
6	qtyReturn	0.0868
7	fanin	0.00761
8	fanout	0.1683
9	age	0.10463
10	nAuth	0.2206
11	nr	0.14766
12	nSmell	0.07386

The most actionable features most correlated with buggyness are LOC (Lines of Code) and fan-out, which measures how many times a method invokes other functions.

The Models

Milestone 3
The refactored Method

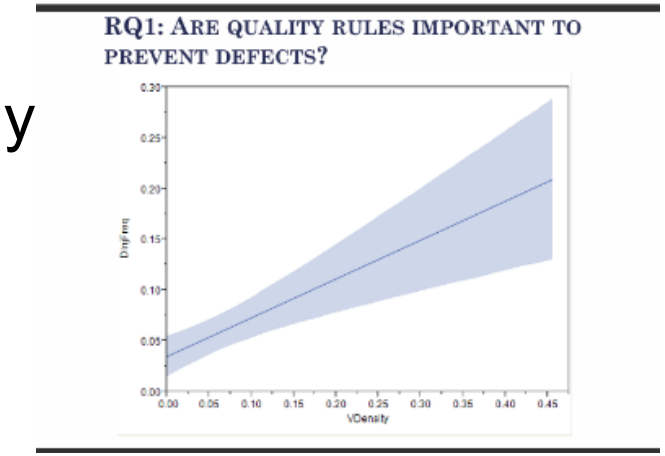
Francesco
Astolfi

Refactored Method (Milestone 3)

1. Rationale for Selecting the Target Method
2. History of the method
3. Description of the selected Method
4. Criteria for identifying Refactoring Targets with the method
5. Resulting refactored method
6. What if analysis

Intro

I chose this method because all the evaluated metrics were significantly high, and after a few releases, the methods underwent substantial changes and no longer exhibited any bugs.
Therefore, I aimed to test the software engineering theory which suggests that a lower number of code smells corresponds to a reduced probability of defects.
The following are the results of this analysis:



release	method	path	loc	age	nAuth	nr	nSmell	buggy
4.1.0	run	hedwig-server/src/main/java/org/apache/hedwig/admin/console/HedwigConsole.java	114	0	1	1	27	True
4.2.0	run	hedwig-server/src/main/java/org/apache/hedwig/admin/console/HedwigConsole.java	39	216	2	7	1	True
4.2.1	run	hedwig-server/src/main/java/org/apache/hedwig/admin/console/HedwigConsole.java	39	286	2	7	1	False

Refactored Methods

HISTORY OF THE METHOD

Release 4.2.0 id=3	
No ticket	
Release 4.2.1 id = 4	
BOOKKEEPER-457	Iv_evaluated=4.1.0 Iv=null Ov=4.2.0 Fv=4.2.0
BOOKKEEPER-439	Iv_evaluated=4.1.0 Iv=null Ov=4.2.0 Fv=4.2.0
BOOKKEEPER-330	Iv_evaluated=4.1.0 Iv=null Ov=4.2.0 Fv=4.2.0
Release 4.2.0 test	
BOOKKEEPER-561	Iv_evaluated=4.1.0 Iv=null Ov=4.2.1 Fv=4.2.1
BOOKKEEPER-557	Iv_evaluated=4.1.0 Iv=null Ov=4.2.1 Fv=4.2.1

As previously observed, we can assume that the method was introduced in release 4.1.0 and remained buggy for the first two subsequent releases. Each bug persisted, on average, for one release cycle—equivalent to approximately 143 days.
This analysis is based on the corresponding issue fix tickets.

	4.0.0	4.1.0	4.2.0	4.2.1	4.2.2[test]
Train 2 [4.1.0]	not exist				
Train 3 [4.2.0]	not exist	nb			
Train 4 [4.2.1] ticket 457,439 e 330	not exist	b	nb		
Test 5 [4.2.2] ticket 561 e 557	not exist	b	b	nb	

Link to the document

Refactored Methods

DESCRIPTION OF THE METHOD:

The method exposes an interface for executing commands on a console.
It supports execution both through the JLine library and via a simpler mode that does not rely on it.
This design choice support the method's usability, making it more userfriendly.
At first examining the method, what stood out was the complexity introduced by an undefined number of catch blocks, which made it difficult to read.
Although the associated metric of trycatch value may appear low, it is actually among the highest compared to other methods in the BookKeeper project.

Selected attribute
Name: qtyTryCatch Type: Numeric
Missing: 0 (0%) Distinct: 6 Unique: 0 (0%)

Statistic	Value
Minimum	0
Maximum	5
Mean	0.16
StdDev	0.505

release	method	loc	wmc	assignments Qty	mathOperati onsQty	**qtyTry Catch**	qtyReturn	fanin	fanout	age	nAuth	nr	nSmell	buggy
4.01.00	run	114	20	37	1	**3**	0	1	7	0	1	1	27	VERO
4.02.00	run	39	7	9	1	**1**	0	1	7	216	2	7	1	VERO
4.02.01	run	39	7	9	1	**1**	0	1	7	286	2	7	1	FALSO

Refactored Methods

Criteria for the refactor:

I chose to refactor this method also because it provided an opportunity to observe how it was refactored in the subsequent version.
The method showed a significant reduction across several metrics; however, it was not possible to simply copy and paste the refactored code, as the changes were not straightforward.
Although the core functionality remained the same, there was no clearly defined or transparent refactoring process.

release	method	loc	wmc	assignmentsQty	mathOperationsQty	qtyTryCatch	qtyReturn	fanin	fanout	age	nAuth	nr	nSmell	buggy
4.01.00	run	114	20	37	1	3	0	1	7	0	1	1	27	VERO
4.02.00	run	39	7	9	1	1	0	1	7	216	2	7	1	VERO

Refactored Methods

Criteria for the refactor:

For the refactoring process, I chose to divide the method into multiple smaller functions, each with its own specific responsibility.
The intention was to potentially increase the overall complexity metrics at the class level, while reducing the complexity values for each individual method. Create a class that best represents the domain of a console with terminal, commands, chronology.

[Refactor from AI assistant in IntelliJ]

Refactored Methods

Resulting refactored method:

For the refactoring, I chose to subdivide the method into multiple functions, each with its own specific responsibility.
The expectation was to increase the overall complexity metrics while reducing the complexity

	loc	wmc	assignments Qty	math OperationsQty	qtyTryCatch	qtyReturn	fanin	fanout	nSmell
Totals	114	24	24	0	2	3	17	23	23
Average	7,6	1,6	1,6	0	0,133	0,2	1,133	1,533	1,533

release	method	className	loc	wmc	assignmentsQty	mathOperationsQty	qtyTryCatch	qtyReturn	fanin	fanout	nSmell	buggy
4.01.00	runJLineConsole	HedwigConsole	11	2	1	0	1	0	1	4	1	False
4.01.00	executeShutdownSequence	HedwigConsole	10	2	1	0	1	0	1	2	1	False
4.01.00	runInteractiveMode	HedwigConsole	8	2	0	0	0	0	1	3	0	False
4.01.00	runSimpleConsole	HedwigConsole	7	2	2	0	0	0	2	1	1	False
4.01.00	executeLine	HedwigConsole	8	2	0	0	0	0	3	3	0	False
4.01.00	run_refactored	HedwigConsole	7	2	0	0	0	0	0	4	1	False
4.01.00	isJLineAvailable	HedwigConsole	11	2	0	0	1	2	1	0	0	False
4.01.00	initializeConsole	HedwigConsole	5	1	2	0	0	0	1	1	0	False
4.01.00	loadHistoryEntries	HedwigConsole$ConsoleHandler	14	3	4	0	0	0	1	1	6	False
4.01.00	initializeCommandMethods	HedwigConsole$ConsoleHandler	4	1	2	0	0	0	1	0	2	False
4.01.00	setupCompletor	HedwigConsole$ConsoleHandler	6	1	3	0	0	0	1	0	4	False
4.01.00	getHistoryFile	HedwigConsole$ConsoleHandler	6	1	2	0	0	1	1	0	0	False
4.01.00	initializeConsoleReader	HedwigConsole$ConsoleHandler	5	1	2	0	0	0	1	0	2	False
4.01.00	runCommandLoop	HedwigConsole$ConsoleHandler	7	2	1	0	0	0	1	2	0	False
4.01.00	setupJLineConsole	HedwigConsole$ConsoleHandler	6	1	0	0	0	0	1	4	0	False
4.01.00	setupHistory	HedwigConsole$ConsoleHandler	10	1	5	0	0	0	1	2	6	False
		Totali	114	24	24	0	2	3	17	23	23	
		Medie	7,6	1,6	1,6	0	0,133333	0,2	1,133333	1,533333	1,533333	

Refactored Methods

INTRO

I chose this method because all the evaluated metrics were significantly high, and after several releases, the method underwent complete changes and no longer exhibited any bugs.
Analyzing this method provided me with an opportunity to gain a deeper understanding of the historical development within my project.

release	method	path	loc	age	nAuth	nr	nSmell	buggy
0.09.00	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	212	45	2	8	14	False
0.09.06	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	212	185	3	9	14	False
0.09.07	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	213	334	1	3	14	True
1.00.00	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	213	457	2	2	14	True
1.00.01	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	213	529	1	2	14	True
1.00.02	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	232	629	2	6	14	True
1.01.00	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	3	723	4	8	0	True
1.00.03	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	3	785	1	1	0	False
1.02.00	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	3	804	1	1	0	False
2.0.0-M1	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	3	974	1	6	0	False
1.02.01	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	3	1022	3	7	0	False
2.0.0-M2	eval	openjpa-kernel/src/main/java/org/apache/openjpa/kernel/jpql/JPQLExpressionBuilder.java	268	1098	4	13	14	False

History of the method

Release 4.2.0 id=3	
No ticket	
Release 4.2.1 id = 4	
BOOKKEEPER-457	Iv_evaluated=4.1.0 Iv=null Ov=4.2.0 Fv=4.2.0
BOOKKEEPER-439	Iv_evaluated=4.1.0 Iv=null Ov=4.2.0 Fv=4.2.0
BOOKKEEPER-330	Iv_evaluated=4.1.0 Iv=null Ov=4.2.0 Fv=4.2.0
Release 4.2.0 test	
BOOKKEEPER-561	Iv_evaluated=4.1.0 Iv=null Ov=4.2.1 Fv=4.2.1
BOOKKEEPER-557	Iv_evaluated=4.1.0 Iv=null Ov=4.2.1 Fv=4.2.1

As seen before we can assume that the method was introduced in even before the 0.9.0 born as Non Buggy but then from 0.9.7 to 1.1.0 result as buggy. And in the future is no more buggy, for simplicity there is a report just until the release 1.2.0 but even in the successive release is no more buggy. The max release of snoring is for 2 release.
Ex : 1.0.0-1.0.2.

Link to the document

	0.9.0	0.9.6	0.9.7	1.0.0	1.0.1	1.0.2	1.1.0	1.0.3	1.2.0
Train 2 [0.9.6]	nb								
Train 3 [0.9.7]	nb	nb							
Train 4 [1.0.0]	nb	nb	nb						
Train 5 [1.0.1] ticket 244	nb	nb	b	nb					
Train 6 [1.0.2]	nb	nb	b	nb	nb				
Train 7 [1.1.0] ticket 420	nb	nb	b	b	b	nb			
train 8 [1.0.3] ticket 586 & 547	nb	nb	b	b	b	b	b		
train 9 [1.2.0] ticket 495	nb	nb	b	b	b	b	b	nb	

Refactored Methods

OpenJPA

Description of the method:

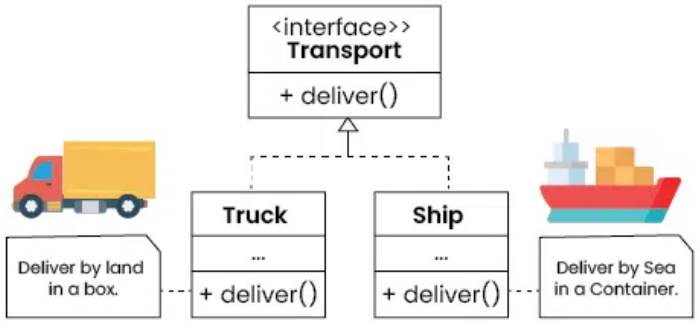
The method expose an interface for query written in JPQL a java persistence query language and it converts it to an expression so that the program could execute it. The first thing that impress my self is the complexity of handle all the method by switch statement, not evaluated metrics in actual condition.

release	method	loc	wmc	assignmentsQty	mathOperationsQty	qtyTryCatch	qtyReturn	fanin	fanout	nSmell
0.09.00	eval	212	81	61	2	0	56	12	55	14
0.09.06	eval	212	81	61	2	0	56	11	33	14
0.09.07	eval	213	83	61	2	0	57	12	55	14
1.00.00	eval	213	83	61	2	0	57	12	55	14
1.00.01	eval	213	83	61	2	0	57	12	55	14
1.00.02	eval	232	88	68	5	0	60	12	50	14
1.01.00	eval	3	2	0	0	0	1	1	0	0
1.00.03	eval	3	2	0	0	0	1	1	0	0
1.02.00	eval	3	2	0	0	0	1	1	0	0
2.0.0-M1	eval	3	2	0	0	0	1	1	0	0
1.02.01	eval	3	2	0	0	0	1	1	0	0
2.0.0-M2	eval	268	109	68	2	0	77	12	50	14

Refactored Methods

Criteria for the refactor:

I chose to refactor in order to decompose the code and adopt a more modular approach.
From this analysis, I also decided to extend the analysis to the class level, taking into account nested inner classes within the same file.
The refactoring was based on applying a registry pattern, where a factory uses a JPQL node to return the corresponding JPQL expression.
[Refactor generated by Deepseek] I chose to refactor in order to decompose the code and adopt a more modular approach.From this analysis, I also decided to extend the analysis to the class level, taking into account nested inner classes within the same file.The refactoring was based on applying a registry pattern, where a factory uses a JPQL node to return the corresponding JPQL expression.[Refactor generated by Deepseek]



It can handle more workload that oder ia wont to elaborate

Refactored Methods

OpenJPA

Resulting refactored method:

For the refactoring, I chose to subdivide the method into multiple functions, each assigned a specific responsibility.
The expectation was to increase the overall complexity metrics while reducing the complexity values of the individual methods.

release	method	className	loc	wmc	assignmentsQty	mathOperationsQty	qtyTryCatch	qtyReturn	fanin	fanout	nSmell
			…								
0.09.07	evaluate	JPQLExpressionBuilder$NotEvaluator	3	1	0	0	0	1	0	3	0
0.09.07	evaluate	JPQLExpressionBuilder$TrimEvaluator	14	9	7	0	0	1	0	8	0
0.09.07	evaluate	JPQLExpressionBuilder$LocateEvaluator	10	4	4	0	0	1	0	11	0
0.09.07	setImplicitTypes	JPQLExpressionBuilder	13	12	5	0	0	4	2	3	2
0.09.07	getIdentifier	JPQLExpressionBuilder	17	4	4	0	0	3	2	5	0
0.09.07	getParameter	JPQLExpressionBuilder$EvaluationContext	3	1	0	0	0	1	1	1	0
0.09.07	evaluate	JPQLExpressionBuilder$LikeEvaluator	11	3	5	0	0	2	0	9	2
0.09.07	supports	JPQLExpressionBuilder$ComparisonOperatorEvaluator	3	7	0	0	0	1	0	0	0
0.09.07	supports	JPQLExpressionBuilder$BetweenEvaluator	3	2	0	0	0	1	0	0	0
		Total	792	348	135	10	3	199	162	296	46
		Average	7,135135	3,135135	1,216216	0,09009	0,027027	1,792793	1,459459	2,666667	0,414414
0.09.07	eval	JPQLExpressionBuilder	213	83	61	2	0	57	12	55	14

The old method pre-refactor

Refactored Methods

OpenJPA

Milestone 4
What if Analysis

Francesco Astolfi

What if analysis

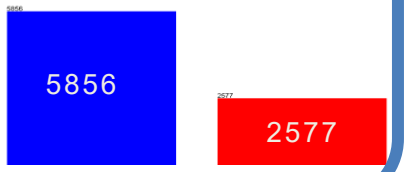
Your Project (2/2)

10. Create:
    1. B+: Portion of A with NSmells >0
    2. C portion of A with NSmells =0.
    3. B: B+ manipulated with feature NSmells set to 0
11. Train BClassifier on A, aka BClassifierA
12. Predict A, B, B+, C and create a Table like in slide 14
13. Analyze the Table and answer: How many buggy methods could have been prevented by having zero smells?
    10. In total
    11. In proportion
    12. Out of the preventable ones

What if 0 smells?

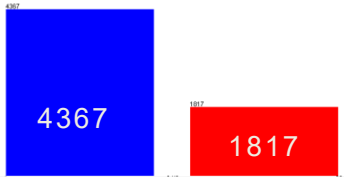
Actual

A Matrix



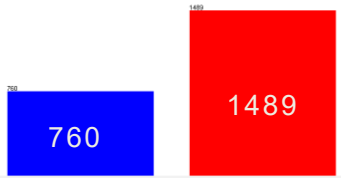
2577 actual bug

C Matrix



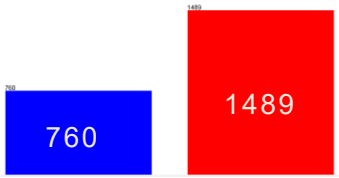
1817 actual bug
smell==0

B+ Matrix



1489 actual bug
smell>0

B Matrix



1489 actual bug
smell set to 0

Random Forest Trained on A

A Matrix

```
=== Confusion Matrix ===

    a    b   <-- classified as
 5822   34 |    a = false
   29 2548 |    b = true
```

2552
predicted bug

C Matrix

```
=== Confusion Matrix ===

    a    b   <-- classified as
 4335   32 |    a = false
   28 1789 |    b = true
```

1821
predicted bug

B+ Matrix

```
=== Confusion Matrix ===

    a    b   <-- classified as
  758    2 |    a = true
    1 1488 |    b = false
```

1490
predicted bug

B Matrix

```
=== Confusion Matrix ===

    a    b   <-- classified as
  758    2 |    a = true
    1 1488 |    b = false
```

1490
predicted bug

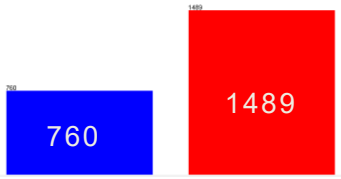
What if analysis

Experimental learning:

Link to the document

	BOOKKEEPER	Smell Correlation	Bug Correlation
1	loc	0.69269	0.09747
2	wmc	0.57613	0.04225
3	assignmentQTY	0.58412	0.04935
4	mathOpQTY	0.41173	0.07979
5	TryCatchQTY	0.3765	0.0954
6	qtyReturn	0.14486	0.0868
7	fanin	-0.00431	0.00761
8	fanout	0.26399	0.1683
9	age	0.0151	0.10463
10	nAuth	0.01244	0.2206
11	nr	0.02656	0.14766
12	nSmell		0.07386

B Matrix



1489 actual bug
smell set to 0

B Matrix

```
=== Confusion Matrix ===

   a    b   <-- classified as
 758    2 |    a = true
   1 1488 |    b = false
```

1490
predicted bug

In the Bookkeeper dataset, the model is able to accurately predict bugs even in the absence of smells. In fact, performance slightly improves without them — for example, in the C matrix (where smell = 0), the model misclassifies only 3 bugs compared to the original A matrix.

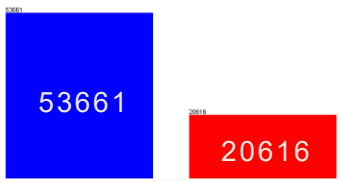
As a result we can see that there is a low correlation between smell and bug, but also that all the strong correlated feature with the nSmell are not so much strong also for the bug prediction maybe this is also due to the fact that the number of instance is to low for training (the model not learned the correlation between loc and buggyness as we generally expected)

What if analysis

What if O smells?

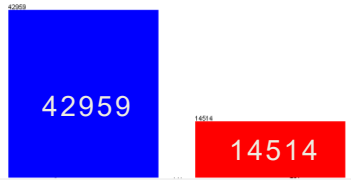
Actual

A Matrix



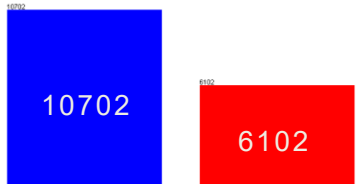
20616 actual bug

C Matrix



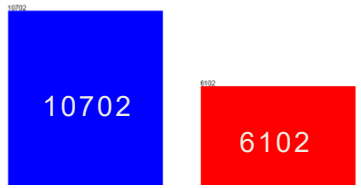
14251 actual bug
smell==0

B+ Matrix



6102 actual bug
smell>0

B Matrix



6102 actual bug
smell set to 0

Random Forest Trained on A

A Matrix

```
=== Confusion Matrix ===

     a     b   <-- classified as
 53476   185 |     a = false
   263 20353 |     b = true
```

20538
predicted bug

C Matrix

```
=== Confusion Matrix ===

     a     b   <-- classified as
 39824  3135 |     a = false
  4519  9995 |     b = true
```

13130
predicted bug

B+ Matrix

```
=== Confusion Matrix ===

     a     b   <-- classified as
 10699     3 |     a = false
     7  6095 |     b = true
```

6098
predicted bug

B Matrix

```
=== Confusion Matrix ===

    a    b   <-- classified as
 9960  742 |     a = false
 1154 4948 |     b = true
```

5690
predicted bug

What if analysis

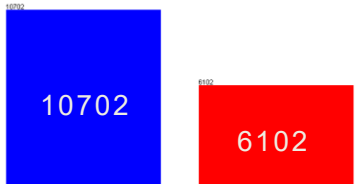
OpenJPA

Lesson learned :

[Link to the document]

OPENJPA		Smell Correlation	Bug Correlation
1	loc	0.5847	0.1386
2	wmc	0.4341	0.1221
3	assignmentQTY	0.4538	0.1061
4	mathOpQTY	0.2166	0.0463
5	TryCatchQTY	0.1635	0.0737
6	qtyReturn	0.174	0.0868
7	fanin	0.0135	0.0847
8	fanout	0.3762	0.1683
9	age	0.0217	0.1195
10	nAuth	0.041	0.2574
11	nr	0.0568	0.4395
12	nSmell		0.0731

B
Matrix



6102 actual bug
smell set to 0

B
Matrix

```
=== Confusion Matrix ===

    a    b   <-- classified as
 9960  742 |    a = false
 1154 4948 |    b = true
```

5690
predicted bug

The number of bug that could be removed without the smell is:

6102 – 5690 = 412 the (7.2%)

[Actual] – [Predicted with 0 smell]

OpenJPA shows a relatively low injection of code smells. Only about 7.2% of bugs appear to be correlated with the presence of smells.

Note: This is an approximate analysis, as no normalization or filtering was applied to reduce the influence of features correlated with buggyness (e.g., LOC). In particular, large methods with hundreds of lines can still be marked as "0 smell" but be highly defect-prone.

What if analysis

OpenJPA